Validierung und Unterricht

Lernunterlage: Was aus dem Unterricht vom 30.10. in Home Control steckt, und wie

1 Eingebaute Validierung (.NET 10)

Minimal APIs nehmen ohne Validierung jedes JSON an. Ein leeres `{}` ergibt ein Objekt mit `null`-Feldern, obwohl `string` nicht nullable ist: Nullable-Referenztypen prüft nur der Compiler, nicht der JSON-Deserialisierer.

```
// Contracts/AuthContracts.cs – die Regeln stehen am DTO
public record LoginRequest(
    [Required(ErrorMessage = "Bitte eine E-Mail-Adresse eingeben.")]
    [EmailAddress(ErrorMessage = "Bitte eine gültige E-Mail-Adresse eingeben.")]
    [StringLength(256, ErrorMessage = "...")]
    string Email,
    [Required(ErrorMessage = "Bitte ein Passwort eingeben.")]
    [StringLength(128, ErrorMessage = "...")]
    string Password);

// Program.cs – einschalten
builder.Services.AddProblemDetails();
builder.Services.AddValidation();
```

- **Vor dem Handler:** Ist die Eingabe ungültig, antwortet die API mit **400** und ProblemDetails (`errors` je Feld). Der Endpunkt läuft gar nicht erst.
- **Deutsche Meldungen:** über `ErrorMessage`. Standard wäre Englisch („The Email field is required.“).
- **Rolle:** `[RegularExpression("^(Admin|User)$")]` statt eigener if-Abfrage.
- **Was bleibt im Code:** Regeln, die Daten brauchen (E-Mail schon vergeben) und die Passwortregeln. Die prüft Identity beim Anlegen.

> **Beim Login keine Passwortregeln**, nur Pflicht und Obergrenze. Sonst verrät die Antwort etwas über die Regeln statt nur „E-Mail oder Passwort ist falsch“.

Vorher und nachher

Vorher	Jetzt
Eigene Methode ValidateEmailAndPassword (35 Zeilen) im Endpunkt	Attribute am DTO, AddValidation prüft automatisch
if (request.Role is not null && ...) errors["role"] = ...	[RegularExpression("^(Admin|User)$")]
Jeder Endpunkt ruft die Prüfung selbst auf	Gilt für jeden Endpunkt, der das DTO annimmt

Testen

`ValidationTests` schickt ungültige Anfragen an die echte API (WebApplicationFactory) und prüft 400 und die deutschen Meldungen. Fallstrick: System.Text.Json schreibt Umlaute im JSON als `\u00FC`. Deshalb liest der Test die Meldungen mit JsonDocument aus, statt im Text zu suchen.

2 GetUserId(): Wer fragt an?

```
public static Guid? GetUserId(this ClaimsPrincipal user) =>
    Guid.TryParse(user.FindFirst(JwtRegisteredClaimNames.Sub)?.Value, out var id) ? id : null;

// im Endpunkt
if (user.GetUserId() is not { } userId)
    return Results.Problem(statusCode: 401, title: "Ungültiges Token.");
```

- **Erweiterungsmethode** (`this ClaimsPrincipal`): liest sich wie eine eingebaute Methode und steht an einer Stelle statt in jedem Endpunkt.
- **Grundsatz aus dem Unterricht:** Wer anfragt, steht im Token (`sub`), nie im Body. Sonst könnte jeder im Namen anderer handeln.
- **null statt Ausnahme:** Der KI-Agent meldet sich per API-Schlüssel an und hat keine Benutzer-Id. Dann gibt es 401 und keinen Absturz.

3 OpenAPI-Angaben für Scalar

```
var rooms = app.MapGroup("/api/rooms").WithTags("Räume");
auth.MapPost("/login", Login).AllowAnonymous()
    .WithSummary("Anmelden: Access Token im JSON, Refresh-Token im HTTP-only Cookie");
```

`WithTags` gruppiert die Endpunkte in der API-Doku (/scalar): Räume, Geräte, Ereignisse, Reports, Tuya, Blink, Anmeldung, Status. `WithSummary` schreibt einen Satz dazu. Das ist praktisch für die Präsentation: Die Doku erklärt sich selbst.

4 Vergleich mit dem Unterricht (Stand 31.10.2026)

Thema	Unterricht (Blog-API)	Home Control	Ergebnis
Benutzer	ASP.NET Core Identity	ASP.NET Core Identity	gleich
Passwort-Hash	PBKDF2 über Identity	PBKDF2 über Identity	gleich
Sperre nach Fehlversuchen	nein (CheckPasswordAsync zählt nicht)	ja: 5 Fehler, 5 Minuten (AccessFailedAsync)	Home Control weiter
Validierung	AddValidation + DataAnnotations	AddValidation + DataAnnotations, deutsch	gleich
User-Id aus Token	GetUserId()	GetUserId()	gleich
OpenAPI-Angaben	WithTags, WithSummary	WithTags, WithSummary	gleich
Token-Laufzeit	60 Minuten, kein Refresh	15 Minuten + Refresh-Token mit Rotation	Home Control weiter
Rechte	Besitz (nur der Autor)	Rolle (Admin) + API-Schlüssel nur lesend	anderer Ansatz
Registrierung	offen	erster Benutzer Admin, danach nur Admin	Home Control weiter
Rückgabetypen	TypedResults + Results<…>	IResult + Results.…	Unterricht weiter
Endpunktfilter	RejectEmptyIdFilter, TimingFilter	keine	Unterricht weiter

Thema	Unterricht (Blog-API)	Home Control	Ergebnis
Tests	Postman von Hand	113 xUnit-Tests, ein Befehl: ./pruefen.sh	Home Control weiter

Noch offen: TypedResults (Umbau aller Endpunkte, gut für einen eigenen Pull Request) und ein Endpunktfilter für leere GUIDs. Beides steht im Übergabe-Dokument.

5 Kurzantworten für Rückfragen

Frage	Antwort
Wann läuft die Validierung?	Nach dem Lesen des Bodys, vor dem Endpunkt. Bei Fehlern läuft der Endpunkt nicht.
Warum [Required], obwohl string nicht nullable ist?	Nullable prüft nur der Compiler. Fehlt das Feld im JSON, ist es zur Laufzeit trotzdem null.
Was gehört nicht in Attribute?	Regeln, die Daten brauchen (E-Mail vergeben?). Die prüft der Service, Antwort ebenfalls als ValidationProblem.
Warum GetUserId() als Erweiterung?	Einmal geschrieben, überall gleich benutzt, und der Endpunkt bleibt lesbar.

6 Merksätze

- Regeln ans DTO, AddValidation prüft vor dem Endpunkt. Ungültig → 400.
- Meldungen für Benutzer auf Deutsch (ErrorMessage).
- Regeln mit Datenbankzugriff im Service, nicht in Attributen.
- Wer anfragt, steht im Token: user.GetUserId().
- WithTags/WithSummary: Die API-Doku erklärt sich selbst.